

p0966R1

`string::reserve` Should Not Shrink

Published Proposal, 8 February 2018

This version:

<http://wg21.link/P0966R1>

Authors:

[Mark Zeren](#) (VMware)

[Andrew Luo](#) (Self)

Audience:

LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

<https://github.com/mzeren-vmw/iso/blob/master/reserve/p0966r1.bs>

Abstract

`basic_string::reserve` should not shrink-to-fit.

Table of Contents

1	History
1.1	Changes from R0
2	Summary
3	Proposed Wording
4	Discussion
4.1	<code>vector::reserve</code>
4.2	String reference invalidation

References

Informative References

§ 1. History

§ 1.1. Changes from R0

- Deprecated default argument of `reserve`, as suggested by LWG in Jacksonville 2018.
- LEWG Jacksonville 2018 accepted without edits:

SF	F	N	A	SA
13	10	0	0	0

- Rebased on [\[N4727\]](#).
- Fixed URLs and typos.

§ 2. Summary

`basic_string::reserve` optionally shrinks to fit `[string.capacity]`:

11. Effects: After `reserve()`, `capacity()` is greater or equal to the argument of `reserve`. [Note: Calling `reserve()` with a `res_arg` argument less than `capacity()` is in effect a non-binding shrink request. A call with `res_arg <= size()` is in effect a non-binding shrink-to-fit request. — end note]

Optionally shrinking-to-fit is problematic because:

- It is a **performance trap**. The authors independently encountered code where the shrink-to-fit behavior added unexpected and costly dynamic reallocations.
- It is a **portability barrier**. Because shrink-to-fit is *optional*, applications may have different behavior when run with one standard library implementation vs. another. The original LWG issue, [\[LWG2968\]](#), was motivated by a performance degradation encountered while porting from MSVC/Dinkumware to GCC's libstdc++.
- It **complicates generic code** - Generic code which accepts `vector` or `basic_string` as a template argument and that is sensitive to shrink-to-fit must add code to avoid calling

`reserve(n)` when `n` is less than `capacity`.

- It **duplicates functionality** available in `basic_string::shrink_to_fit`.
- It is **inconsistent** with `vector::reserve` which does not shrink-to-fit.

§ 3. Proposed Wording

We propose resolving [\[LWG2968\]](#) by:

1. Overloading `basic_string::reserve` in order to deprecate its default argument.
2. Changing the allocation behavior of `basic_string::reserve` to mirror `vector::reserve`.

Wording is relative to [\[N4727\]](#).

Change 24.3.2.4 [**basic.string**] as depicted:

```
// 24.3.2.4, capacity
size_type size() const noexcept;
size_type length() const noexcept;
size_type max_size() const noexcept;
void resize(size_type n, charT c);
void resize(size_type n);
size_type capacity() const noexcept;
void reserve(size_type res_arg = 0);
void reserve(size_type res_arg);
void shrink_to_fit();
void clear() noexcept;
[[nodiscard]] bool empty() const noexcept;
```

Change 24.3.2.4 [**string.capacity**] as depicted:

~~void reserve(size_type res_arg=0);~~

- ~~10. The member function `reserve()` is a directive that informs a `basic_string` object of a planned change in size, so that it can manage the storage allocation accordingly.~~
- ~~11. *Effects:* After `reserve()`, `capacity()` is greater or equal to the argument of `reserve`. [
Note: Calling `reserve()` with a `res_arg` argument less than `capacity()` is in effect a non-binding shrink request. A call with `res_arg <= size()` is in effect a non-binding shrink-to-fit request. —end note]~~

void reserve(size_type res_arg);

10. *Effects:* A directive that informs a `basic_string` of a planned change in size, so that it can manage the storage allocation accordingly. After `reserve()`, `capacity()` is greater or equal to the argument of `reserve` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of `reserve()`.
11. ~~12~~ *Throws:* `length_error` if `res_arg > max_size()`.²³¹

Add a new section:

D. Deprecated basic_string capacity [depr.string.capacity].

The following member is declared in addition to those specified in [string.capacity]

```
namespace std {  
    template<class charT, class traits = char_traits<charT>,  
            class Allocator = allocator<charT>>  
    class basic_string {  
    public:  
        void reserve();  
    };  
}  
  
void reserve();
```

1. Effects: After this call, capacity() has an unspecified value greater than or equal to size(). [Note: This is a non-binding shrink to fit request. – end note]

§ 4. Discussion

§ 4.1. `vector::reserve`

For reference, here is the Effects section for `vector::reserve`. Emphasis added to highlight the portion used in the new proposed wording for `basic_string::reserve`. See [vector.capacity].

3. Effects: A directive that informs a vector of a planned change in size, so that it can manage the storage allocation accordingly. **After reserve(), capacity() is greater or equal to the argument of reserve if reallocation happens; and equal to the previous value of capacity() otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of reserve().** If an exception is thrown other than by the move constructor of a non-CopyInsertable type, there are no effects.

§ 4.2. String reference invalidation

While the proposed wording makes `basic_string` behave like `vector` in one respect, string iterator and reference invalidation remains distinctly different. See `[string.require]`:

4. References, pointers, and iterators referring to the elements of a `basic_string` sequence may be invalidated by the following uses of that `basic_string` object:
 - (4.1) as an argument to any standard library function taking a reference to non-const `basic_string` as a `n` argument.
 - (4.2) Calling non-const member functions, except `operator[]`, `at`, `data`, `front`, `back`, `begin`, `rbegin`, `end`, and `rend`.

Since C++11 effectively removed support for copy on write strings with [\[N2668\]](#), we believe that `basic_string` can and should evolve to have reference and iterator invalidation more like `vector`. However, that larger edit is not necessary to resolve the current LWG issue

§ 4.3. Code example

Here is a pseudo-code example drawn from real world experience that shows how shrink-to-fit can lead to unexpected allocations.

Imagine the following formatting utility:

```
using std::string;
using std::string_view;

void Append(string& out, string_view arg1, string_view arg2, string_view arg3)
{
    out += arg1;
    out += arg2;
    out += arg3;
}
```

Profiling will show that `Append` can cause as many as three string reallocations. The obvious solution is to add a call to `reserve`:

```

void Append(string& out, string_view arg1, string_view arg2, string_view arg3)
{
    out.reserve(out.size() + arg1.size() + arg2.size() + arg3.size());
    out += arg1;
    out += arg2;
    out += arg3;
}

```

Profiling will show that there are now fewer allocations, but still an unexpected number.

One of the callers of `Append` that shows higher allocation rate looks like this:

```

using std::vector;

struct Element { string a; string b; string c; }

string Serialize(const vector<Element>& elements)
{
    string result = kPrologue;
    for (auto& element : elements) {
        Append(result, element.a, element.b, element.c);
        result += kDelimiter; // <<--- can reallocate
    }
    result += kEpilogue;
    return result;
}

```

What's happening is that `result += kEpilogue;` can grow `result` using the string's normal expansion ratio. Then the next `Append` will shrink `result`.

We can fix that by modifying `Append` once more:

```

void Append(string& out, string_view arg1, string_view arg2, string_view arg3)
{
    string::size_type len = out.size() + arg1.size() + arg2.size() + arg3.size();
    if (len > out.capacity()) {
        out.reserve(len);
    }
    out += arg1;
    out += arg2;
    out += arg3;
}

```

This proposal would remove the need for this last version.

For completeness we note that for both `basic_string` and `vector`, `reserve` bypasses the normal growth algorithm which can also lead to unnecessary allocations. Resolving that issue is out of scope for this proposal.

§ References

§ Informative References

[LWG2968]

Andrew Luo. [Inconsistencies between basic_string reserve and vector/unordered_map/unordered_set reserve functions](https://wg21.link/lwg2968). LEWG. URL: <https://wg21.link/lwg2968>

[N2668]

A. Meredith, H. Boehm, L. Cowl, P. Dimov, D. Krügler. [Concurrency Modifications to Basic String](https://wg21.link/n2668). 12 June 2008. URL: <https://wg21.link/n2668>

[N4727]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4727). URL: <https://wg21.link/n4727>